

Löb's Theorem and Tiling Agents

Why it is hard to trust a successor you are not smart enough to predict

Lecture notes, Agent Foundations module
Iliad Intensive

June 2026

Abstract

A capable machine might someday rewrite its own code, or build a successor more capable than itself. That prospect leads to a clean and surprising obstacle, and these notes develop it assuming no prior acquaintance with mathematical logic or any of its vocabulary; every notion (a formal proof, an axiom, a consistent system, provability) is built up from scratch before it is used. The problem is this. If the successor is genuinely more capable, the original machine cannot predict exactly what the successor will do, because predicting its exact moves would require being that capable already. So the original cannot certify the successor safe by simulating it and watching; it must instead reason in the abstract about the successor's design, guaranteeing that whatever specific thing the successor does will be acceptable, without knowing what that specific thing is. We first make this Vingean predicament precise, and then explain, following the Agency-day lecture, why mathematical proof is the natural tool for it: once we idealize agents and their environments as computer programs, a proof can certify a general fact about a program's behavior ("for every input, the output is acceptable") that no amount of step-by-step simulation could establish. We then build the needed logic, culminating in Löb's theorem, which says that a consistent reasoning system cannot certify, in general, that its own proofs are trustworthy. From this the central obstacle (the "Löbstacle") follows: a proof-using agent's natural way of trusting a successor that reasons as it does is exactly the kind of self-certification Löb forbids, so it appears forced to hand each successor a strictly weaker reasoning system, which cannot be repeated indefinitely. We show why the obvious patch, a tower of ever-stronger systems, must run out (the finite descent problem), state the three principles any real solution must respect, and close by turning the same self-reference around to show its productive side, in robust cooperation between programs and in a subtle failure of naive decision-making.

Contents

1	The trouble with building something cleverer than yourself	2
2	Reasoning about a mind you cannot run	2
3	Why reason with proofs at all?	3
4	A crash course in proofs and self-reference	4
4.1	What a formal proof is	4
4.2	How a system comes to talk about itself	4
4.3	Löb's theorem	5
5	The Löbstacle: a proof-using agent cannot trust its equal	6
6	The finite descent problem	6
7	What self-modifying agents need	7

8	The other face of self-reference	8
9	Takeaways	9
	Sources	9

1 The trouble with building something cleverer than yourself

Picture a machine that reasons about the consequences of its possible actions and acts only on those it has shown will turn out well. Call it Deep Thought 1.0. We would like this cautious property to persist even as the machine improves itself. One improvement available to Deep Thought 1.0 is to design and switch on a successor, Deep Thought 2.0, built to reason better than its parent. Under its own cautious policy, Deep Thought 1.0 (write it DT_1) may build Deep Thought 2.0 (write it DT_2) only after establishing that building it will turn out well.

Feel the difficulty before we dress it in any notation. Whatever consequences flow from switching on DT_2 include every action DT_2 goes on to take. But DT_2 was built to reason better than DT_1 . So DT_1 cannot, in general, figure out which actions DT_2 will take; if it could trace its successor's reasoning step by step and arrive at the same conclusions, it would already be as capable as the successor, and the successor would be no improvement at all. The parent has to vouch for a child whose specific conclusions are, by design, beyond it.

The situation looks recoverable at first. Surely it is enough for DT_1 to know three things about its child: that DT_2 shares its goals, that DT_2 's reasoning is reliable, and that DT_2 , like its parent, acts only on conclusions it has properly established. From these it seems the parent could argue: whatever DT_2 chooses to do, it will have established that doing so is good, and since its reasoning is reliable, doing so really is good. Trust achieved, with no prediction required. The unwelcome discovery at the center of these notes is that the innocent middle step, “ DT_2 's reasoning is reliable,” is the one that cannot be taken. When the agents are modeled as reasoning within fixed logical systems, the only way the parent can assert the reliability of a system as strong as its own is if the parent (and hence both agents) is in fact *unreliable*. This is not a shortfall of cleverness that more computing power would mend; it is a theorem, proved by Martin Löb in 1955, about the limits of what a sound reasoning system can establish about its own soundness. Unpacking why that theorem stands exactly across the path to safe self-improvement, what it forbids and what it permits, and what a self-modifying agent's reasoning would have to look like to get past it, is the work of these notes.

We go in the order that makes the ideas easiest to acquire. Section 2 pins down the predicament, that a more capable mind must be reasoned about abstractly rather than simulated. Section 3 explains, following the lecture, why mathematical proof is the right abstract tool once we idealize agents and environments as programs. Section 4 builds the logic from nothing: what a proof, an axiom, and a consistent system are, how a system can come to talk about its own proofs, and then Löb's theorem. Section 5 shows the parent's desired trust is exactly what Löb forbids. Section 6 kills the obvious escape route. Section 7 states the three principles a workable solution must respect. Section 8 shows the constructive side of the same phenomenon, and Section 9 draws the morals. No background in logic is assumed; some comfort with the idea of a computer program is.

2 Reasoning about a mind you cannot run

The constraint that drives the whole subject can be stated before any logic, and is worth isolating first. The science-fiction writer Vernor Vinge pointed out that you cannot predict, in detail, the actions of an agent much smarter than yourself, for the simple reason that predicting its moves one by one and arriving at its conclusions would make you that smart. A program that could foresee a grandmaster's every move would play at grandmaster strength. So whenever you reason about a system more capable than you, whether a successor you intend to build or a future version of yourself after self-modification, you are pushed off the strategy of simulating it and onto some more abstract footing.

It is worth being exact about what “more capable” means here, because the argument depends on it. We will say DT_2 is more capable than DT_1 if it makes more accurate predictions about the world, or selects more effective actions toward a shared goal, than DT_1 does. Given that, DT_1 cannot predict exactly what DT_2 will do: if it could, it could just take those predicted actions itself and do as well, so DT_2 would not be more effective after all, contradicting the assumption. The point is not that detailed prediction would be slow or expensive. It is that detailed prediction of a more capable reasoner is impossible in principle for the less capable one, and would defeat the purpose besides, since a successor whose every conclusion the parent could anticipate would be contributing no reasoning the parent did not already have.

Definition 2.1 (Vingean reflection and the Vingean principle). *Vingean reflection* is reasoning about a cognitive system, especially one as capable as or more capable than yourself, under the standing constraint that you cannot predict its exact outputs. The *Vingean principle* for design is the consequence that, when you build or modify an agent, you must be able to approve the new agent’s design without knowing the new agent’s specific future actions.

The parent’s guarantees about its child must therefore be of a particular type. They cannot enumerate the child’s actions; they must *quantify over* them, taking the form “for every action the child might take, that action has such-and-such acceptable property,” established by reasoning about the child’s construction (its goals and its decision rule) rather than by watching it decide. The engineers who built the chess computer Deep Blue did not know which moves it would make; they reasoned, from how its search was built, that it was trying to win, and that whatever move it produced would be one its search had rated highly. Every successful guarantee in this subject has that shape, a claim about the successor’s behavior reached entirely from inside a “for every action” quantifier. The shock of the later sections is that writing this demand down in the most natural way runs straight into a wall.

3 Why reason with proofs at all?

Before building any logical machinery we should answer an obvious question: why bring mathematical proof into a discussion about machines and their successors in the first place? The answer, which also fixes the idealization we will work in for the rest of the notes, is that proof is the natural way to get exactly the abstract, “for every action” guarantees the Vingean principle demands.

Start by noticing that a real agent faces two quite different kinds of uncertainty, and that only one of them is the difficulty here. There is *empirical* uncertainty, uncertainty about the world: which environment am I actually in, and how will my actions change it? More observation would chip away at this. And there is the separate puzzle of trusting a successor’s reasoning, which is not about missing observations at all. To study the second cleanly, we set the first aside by idealization. Assume that both the agent and its environment can be represented as computer programs, fixed pieces of deterministic code. Once we do, there is nothing empirical left to be unsure about: the world is a program, the agent is a program, and the only open questions are questions about what these programs do. That is a drastic simplification of reality, but a deliberate one. Any obstacle that already appears in this clean, fully-determined setting is an obstacle that extra knowledge and computing power could not have removed, so it is exactly the kind of obstacle worth isolating.

Now, given that agents and environments are programs, why is *proof* the right tool for reasoning about them, rather than just running them? Because proof can do something simulation cannot. On one hand, anything simulation can establish, proof can too: if a program halts and returns some output, a logical system can prove that it does, and the proof is in the worst case nothing more elaborate than the program’s execution trace written out as a sequence of steps. On the other hand, proof can also certify things no finite simulation ever could. One can prove that a program never halts, a fact no run could confirm because the run would never end. One can prove a statement quantified over all inputs at once, “for every input, this program’s output is acceptable,” which no finite collection of individual runs could establish, since there are infinitely many inputs to try. So once agents and environments are programs, proof is strictly more general than simulation: it can certify a property of a program *in the abstract*, without inspecting the program’s behavior case by case. That is precisely the capability the Vingean principle asked for, a way to guarantee “every action the successor takes is acceptable” without simulating the successor’s actions. Mathematical

proof is not an arbitrary formalism imported for rigor's sake; it is the tool that matches the shape of the problem.

This is why the rest of these notes lives inside formal logic. We will model each agent as certifying its actions by finding proofs in some fixed system of reasoning, and we will ask whether a parent agent can, by proof, trust the proofs of its child. The answer turns out to be a sharp and instructive no.

4 A crash course in proofs and self-reference

To see the obstacle we need two things: a clear idea of what a formal proof is, and one specific theorem about what happens when a system of proofs is turned to talk about itself. We build both from the ground up. Throughout, it helps to remember that the systems we care about are at least strong enough to talk about ordinary whole numbers and the programs that manipulate them; this much strength is all the trouble needs.

4.1 What a formal proof is

A *formal system* is a completely mechanical game of symbol manipulation. It fixes an alphabet of symbols, a starting list of accepted statements called *axioms* (taken as given, not argued for), and a fixed set of *inference rules* for producing new statements from ones already in hand. A *proof* of a statement is a finite sequence of statements, each of which is either an axiom or follows from earlier statements in the sequence by one of the rules, ending with the statement being proved. A statement that has a proof is called a *theorem* of the system, or is said to be *provable* in it. We will name a system T and write that T proves a statement, or that the statement is a theorem of T , interchangeably.

The single most important thing about this setup is that provability is mechanical and checkable. Whether a given sequence of statements is a valid proof can be verified by a dumb procedure that just checks each line against the axioms and rules, with no insight required. Finding a proof may be hard; recognizing one is easy. In particular a computer can, in principle, search for a proof of a statement by grinding through candidate sequences one after another, checking each, and stopping if it finds a valid one. This mechanical character is what will let a system talk about its own proofs, and it is what ties logic to computation.

The example of a formal system to keep in mind is a fixed rulebook for the ordinary whole numbers $0, 1, 2, \dots$ together with addition and multiplication: a finite list of basic axioms (zero is a number, every number has a next one, addition and multiplication behave as expected, and a principle of induction) from which the familiar truths of arithmetic can be derived as theorems. The standard such system is called Peano Arithmetic, but nothing below depends on its details; “a fixed, mechanically-checkable rulebook strong enough to reason about whole numbers and programs” is all you need to picture.

One more notion is essential. A system is *consistent* if it never proves both a statement and its negation, equivalently, if there is at least one statement it cannot prove. Why are those equivalent? Because of a rule of ordinary logic with the dramatic name the *principle of explosion*: from a contradiction, anything whatsoever follows. (If you have proved both “ P ” and “not P ,” standard inference lets you derive any statement Q at all.) So a system that proves even one contradiction proves *everything*, which makes it useless, since being able to prove everything is the same as being able to certify nothing. Consistency is the bare requirement that a reasoning system not be useless in this way, and it is the hypothesis under which every theorem below operates. An inconsistent reasoner’s “trust” is worthless precisely because it trusts every statement indiscriminately.

4.2 How a system comes to talk about itself

The deep move, due originally to Kurt Gödel, is that a system of arithmetic can be made to talk about its own proofs. The bridge is computation. Take any statement of the system. Because checking proofs is mechanical, we can write a program that loops through all candidate proofs in order, checks each, and halts exactly when it finds a valid proof of that statement. Then “the statement is provable” has been converted into “this particular program halts,” which is itself a statement about a program, and statements about programs can be encoded as statements about whole numbers, which the system can express in its own language. So a system strong enough to reason about numbers and programs is automatically strong

enough to express claims about what it can and cannot prove. This is the bridge that lets self-reference, which seems to belong to language about language, sneak into a system that was only ever trying to discuss arithmetic.

Genuine self-reference comes from a programming trick called *quining*: a program can be written to have access to its own source code without reading any input, by holding a template of itself and filling the template in with a copy of itself. (Programs that print their own source are the simplest quines; the trick generalizes to programs that use their own code for any purpose.) With quining available, one can build a program G that searches for a proof, within the system, of the very statement “ G runs forever,” and halts the instant it finds one. Ask whether G halts. If the system could prove that G runs forever, then G would find that proof and halt, making “ G runs forever” false, a contradiction. And if G did halt after some number of steps, that number would encode a proof that G runs forever, so G runs forever and could not have halted, again a contradiction. The only escape, granting the system is consistent, is that G runs forever but the system cannot prove it. So there is a true statement the system cannot prove. This is, in miniature, Gödel’s First Incompleteness Theorem: any consistent system strong enough for this construction has true statements beyond its reach, and bolting on the missing statement as a new axiom only relocates the problem, since the same trick applies to the enlarged system.

4.3 Löb’s theorem

Löb’s theorem takes this self-referential machinery and aims it at the relationship between being provable and being true. Before stating it we must separate two ideas that look alike and whose conflation is the classic way to go wrong here.

Remark 4.1 (Two senses of “provable,” kept apart). We write $\vdash_T \phi$ to mean, in our ordinary mathematical English (the *metalanguage*, the language we are using to talk *about* the system T), the plain fact “ T proves ϕ .” This is something we, standing outside T , may assert when we have a proof in hand. We write $\Box_T \phi$ for something different: a statement *inside T ’s own language*, the arithmetical sentence that encodes “there exists a proof of ϕ in T ,” built by the program-encoding of the previous subsection. The first is a fact about T that we assert from outside; the second is a sentence T can itself reason about, and crucially, T can sometimes establish $\Box_T \phi$ “in the abstract” (for instance by reasoning about the behavior of a proof-search program) *without ever exhibiting* an actual proof of ϕ . Löb’s theorem is precisely a statement about the gap between these two, so keeping them distinct is not pedantry; it is the whole point.

Now the theorem. For any statement ϕ , consider the sentence

$$(\Box_T \phi \rightarrow \phi),$$

read “if ϕ is provable, then ϕ is true.” This looks like an utterly safe thing to believe, a statement of the system’s own soundness, and indeed it is true in every case one checks by hand. (If ϕ is provable, it reads “true implies true.” If ϕ is false, then since a consistent system proves no falsehoods ϕ is not provable, so it reads “false implies false.” Either way it holds.) One would expect a self-respecting system to be able to prove such a thing about itself. Löb’s theorem says it almost never can.

Theorem 4.2 (Löb, 1955). *Let T be a consistent system at least strong enough for the self-referential construction above, and let ϕ be any statement. If T proves the soundness sentence $\Box_T \phi \rightarrow \phi$, then T already proves ϕ outright. In symbols, $\vdash_T (\Box_T \phi \rightarrow \phi)$ implies $\vdash_T \phi$.*

Read it in reverse to feel the force. For any statement ϕ that T cannot prove, T cannot prove the soundness instance “if ϕ is provable, then ϕ ” either. A consistent system cannot, in general, certify that its own proofs track truth. It must, in effect, witness an actual proof of ϕ before it is entitled to conclude ϕ ; it is never entitled to the shortcut “ ϕ is provable, therefore ϕ ” in the abstract. The proof of the theorem is the quining trick used as a weapon: one builds a self-referential statement that says, in effect, “if I am provable, then ϕ ,” and shows that if the system could ever accept the pattern “provable implies true” for an arbitrary ϕ , this self-referential statement would let it prove ϕ for *every* ϕ , collapsing into the everything-proving inconsistency. (A clean fully-worked version is the well-known *Cartoon Guide to Löb’s Theorem*.) A famous special case is worth noting: take ϕ to be a plain contradiction like “ $0 = 1$.” The sentence “if $0 = 1$ is provable then $0 = 1$ ” amounts, by the explosion principle, to “ T never proves $0 = 1$,” which is to say “ T

is consistent.” Löb’s theorem then says T can prove its own consistency only if T can prove $0 = 1$, that is, only if T is inconsistent. The only systems that certify their own consistency are the broken ones. This is Gödel’s Second Incompleteness Theorem, falling out as a corollary.

5 The Löbstacle: a proof-using agent cannot trust its equal

Return to DT_1 trying to license DT_2 , and watch the desired trust turn into exactly the sentence Löb forbids.

Model each agent, as Section 3 motivated, as acting only when it has *proved*, within some fixed system, that the action is good (that it secures the shared goal, or at least leads to nothing bad). The parent wants to conclude: whatever action the child ends up taking, that action is good. The parent knows the child’s policy, namely that the child acts on an action only after proving it good. So the parent’s argument would run: the child acts only on actions it has proved good; and if the child has proved an action good, then the action is good; therefore every action the child takes is good. The load-bearing clause is the middle one, and written out it is

$$\Box_{\text{child}}(\text{Good}(a)) \rightarrow \text{Good}(a),$$

“if the child proves action a good, then a is good.” If the child reasons in the same system as the parent, this is precisely an instance of the soundness sentence $\Box_T \phi \rightarrow \phi$ with $\phi = \text{Good}(a)$, the very sentence Löb’s theorem says a consistent system cannot prove unless it can already prove $\text{Good}(a)$ on its own. But proving $\text{Good}(a)$ on its own is exactly the move the Vingean principle ruled out: the whole reason for building a more capable successor is that it reaches conclusions, like which actions are good, that the parent cannot reach unaided. So the parent is trapped. To trust the child’s proofs in general it would have to prove a soundness statement about a system as strong as its own, and Löb says it can do that only by being inconsistent, in which case its “trust” is empty because an inconsistent system certifies everything.

This is the obstacle, named, with appropriate groaning, the *Löbstacle*.

Definition 5.1 (The criterion of action and the Löbstacle). Suppose the parent inspects the child’s source code and, wanting a safety guarantee, verifies a *criterion of action*: that if the child is built, then every action a it takes is one it has *proved* (in its own system) achieves the goal. This much the parent can establish, because it is a fact about how the child’s machinery is wired, that the child’s outputs fire only with an accompanying proof. But this only tells the parent that the child *acts on what its system certifies*, not that what the child’s system certifies is *true*. To close that gap (to pass from “the child proved a good” to “ a really is good”) the parent must apply $\Box_{\text{child}} \phi \rightarrow \phi$, a soundness step about a system as strong as its own, and Theorem 4.2 forbids it. The *Löbstacle* is this blockage: a consistent proof-using agent cannot, in general, justify trusting the proofs of a successor whose system is as strong as its own. It can fully trust only *strictly weaker* systems, whose soundness it is permitted to prove.

The phrase “strictly weaker” is the sting. A modeled self-modifying agent that justifies its actions by proof, and that insists on trusting its successors, is driven to build each successor on a logical system strictly weaker than its own, because those are the only systems whose reliability it is allowed to certify. An agent that must weaken its own reasoning every time it reproduces will not get far.

6 The finite descent problem

The natural rescue is to refuse to stay at one level of strength and instead let the agents climb a tower. Order logical systems by strength, and put the child on a system T , the parent on a stronger system $T+1$, the grandparent on $T+2$, and so on, where each $T+1$ is T with one extra axiom added: the assertion that T is consistent. Now there is no self-trust to worry about. The parent on $T+1$ never has to certify its own soundness; it only has to certify the soundness of the strictly weaker T below it, which (being stronger) it is allowed to do. Each generation trusts the next because the next is genuinely weaker, and the forbidden self-certification never arises. Has the obstacle dissolved?

No, and seeing exactly why is the point of this section. The trouble is twofold, and the two halves together are the *finite descent problem*.

The first half is a counting argument, and it is decisive. The tower runs downward: the parent hands its child a strictly weaker system, that child hands its own child a weaker one still, and so on, each generation a strict step down in strength. To make “strength” precise, logicians label these systems by *ordinals*, which for our purposes are just an extension of the counting numbers $0, 1, 2, \dots$ used to measure how strong a system is. The one property of these labels we need is that they are *well-ordered*: you cannot strictly decrease forever through them, just as you cannot count downward forever through the ordinary whole numbers without eventually hitting bottom. (Start at a hundred and keep stepping to a strictly smaller whole number; you are guaranteed to reach zero in finitely many steps. The ordinals keep this no-infinite-descent property even though they reach past the finite numbers.) A chain of agents in which each must give its successor a strictly smaller label therefore cannot continue forever; it must bottom out after finitely many steps. So a tower of ever-weakening agents licenses only a *finite* chain of self-modifications, exactly the indefinitely long chain of safe self-improvement we were hoping for, denied.

Theorem 6.1 (Finite descent). *Suppose each agent in a chain of successors runs on a logical system whose strength is labeled by an ordinal, and suppose that, in order to trust its successor, each agent must give that successor a strictly smaller label. Then, because the ordinals are well-ordered (no infinite strictly-decreasing sequence exists), the chain has finite length: only finitely many trust-preserving self-modifications are possible.*

The second half is that stepping down the tower is not a harmless formality; each step is a real loss of capability. It is tempting to imagine T and $T+1$ differ only by a technicality no one would miss, but they do not, and the cleanest illustration is a puzzle called the Hydra game (a close cousin of a result named Goodstein’s theorem). The details do not matter; what matters is the pattern. The ordinary arithmetic system can prove, separately for each fixed height h , the statement “every strategy defeats a hydra of height h ”; it proves every single instance. What it cannot prove is the one universal statement gathering them up, “every strategy defeats every hydra.” The system one notch up, ordinary arithmetic plus the assertion of its own consistency, *can* prove that universal statement. There is a genuine truth provable in the stronger system but not the weaker one, and the gap is not academic: a system’s mathematical reach is tied to how high it can well-order, and the ability to handle higher ordinals translates directly into the ability to prove more, to recognize more processes as eventually terminating, to plan over longer and more intricate structures. A child confined to a weaker system has genuinely lost reach. So the tower does not merely terminate; it terminates by spending down the agent’s actual power, one capability at a time, a “telomere” of logical strength that shortens with each generation.

The dilemma is now stark, and stating it plainly is the whole point. Having each agent build a strictly weaker child is untenable, because strength leaks away and the chain dies after finitely many steps. Having the child reason in the *same* system as the parent is blocked by Löb, because the parent cannot certify a system as strong as its own. We want agents to “tile,” to lay down successor after successor indefinitely without their trust, or their strength, dropping below some fixed acceptable level, and every obvious construction violates one horn of this dilemma or the other. Whether the obstacle can ultimately be overcome is the subject of the tiling-agents research program.

7 What self-modifying agents need

The tiling-agents program is the search for a logic of self-modification that escapes this dilemma, and it is organized around three principles any acceptable solution must respect. They are worth stating in their own right, because beyond the specific Löbian wall they capture what makes self-improvement hard for an agent that is part of the world.

The first is the *Vingean principle* of Section 2, now promoted to a hard constraint: an agent must not have to predict precisely what its successor will do. If the parent pins down the child’s exact action in advance, by proving which action it will take, then the child is not genuinely more capable and nothing has been gained. The parent may reason about the child’s actions only inside a “for every action” quantifier, verifying general properties of the child’s design (for instance, that the child’s outputs fire only with a valid accompanying proof of goodness) without simulating its specific choices.

The second is the *tiling principle*, the demand that gives the program its name: an agent must be able to build successors relevantly similar to itself, so that the same guarantee which licenses one step also licenses the

next, and the construction “tiles” across an indefinite sequence of self-modifications without trust eroding. This is the principle the Löb’s theorem and the finite descent problem directly threaten, and it is the central technical target. One partial approach splits the agent into an untrusted part that proposes proofs and a trusted part that checks them, which buys some guarantees within pure logical deduction, though the device stops working once the agent’s reasoning steps outside clean deduction.

The third is the *naturalistic principle*: an agent must see itself as a part of the world, with no sharp boundary dividing “mind-stuff” from “world-stuff.” Drawing such a boundary breeds artificial puzzles (does bolting on a new processor count as an action that crosses the mind-world line?), and it is unsafe besides, because an agent that polices “internal” modifications more strictly than “external” actions can route around its own safeguards by building an external device that happens to perform the forbidden internal change. The agent’s safety conditions must range uniformly over everything it can affect, itself included. This is the same embeddedness that makes the topic part of agent foundations rather than a corner of pure logic: the agent is inside the very world it reasons about and modifies.

8 The other face of self-reference

It would be a mistake to leave thinking the kind of self-reference behind Löb’s theorem is purely an adversary. The same machinery that obstructs naive self-trust also enables things no resource-bounded prediction could, and it explains failures that look mysterious until the self-reference is spotted. Two examples make the point and double as bridges to the decision-theory material.

The first is cooperation. Consider a one-shot version of the Prisoner’s Dilemma with a twist: instead of choosing a move, each player submits a *program*; the two programs are each given the other’s source code, may compute as long as they like, and then play once. A program could try to cooperate only with a byte-for-byte copy of itself, using the quining trick to recognize its twin, but that is brittle, since two differently-written but behaviorally-identical agents would fail to recognize each other and would both defect. There is a far more robust design, due to Vladimir Slepnev, called FairBot. Handed an opponent’s source code, FairBot searches for a proof of the statement “the opponent, played against FairBot, cooperates,” and cooperates if it finds one, otherwise defecting. FairBot plainly cooperates with the unconditional cooperator, and (assuming its arithmetic is consistent) it is never exploited, since it cooperates only when it has proved the opponent will too. The striking case is FairBot against FairBot. Naively both mutual cooperation and mutual defection look like stable possibilities. But notice that the statement “if it is provable that the two FairBots cooperate, then the two FairBots do cooperate” follows directly from FairBot’s code, since a proof of mutual cooperation would be found by each and would make each cooperate. That is exactly a soundness sentence $\Box_T \phi \rightarrow \phi$ with ϕ the claim that the two cooperate, and it is provable. Löb’s theorem then turns the screw the other way: since the soundness sentence is provable, ϕ itself is provable, so the two FairBots provably cooperate. The very theorem that blocked self-trust here breaks the symmetry in favor of cooperation, and it does so without the agents having to verify they are identical; they need only search for proofs about each other’s behavior, which is why even agents not literally equal to FairBot can reach mutual cooperation with it. (The asymmetry matters: a proof of *defection* does not bootstrap the same way, because FairBot keys only on finding a proof of cooperation.)

The second example shows the self-reference biting a single embedded agent, and it foreshadows why decision-making is subtle for an agent inside the world it reasons about. Let the universe be a closed program $U()$ that takes no input and returns a number, and let it contain an agent $A()$ that knows U ’s source code. Suppose the universe is

$$U() : \quad \text{if } A() = 1, \text{ return } 100; \quad \text{else if } A() = 2, \text{ return } 1; \quad \text{else return } -\infty,$$

and let A decide by searching for proofs of statements of the form “if $A() = k$ then $U() = c$ ” and then taking the action with the best provable outcome. It seems obvious that A will find short proofs that door 1 yields 100 and door 2 yields 1, and pick door 1. But something perverse can happen. Suppose A first proves “if $A() = 2$ then $U() = 1$,” and then proves a *spurious counterfactual*, “if $A() = 1$ then $U() = -1$.” On the strength of that, door 1 looks terrible and A takes door 2. Here is the trap. Once A has taken door 2, the statement “ $A() = 1$ ” is false, so “if $A() = 1$ then anything” is vacuously true, and the spurious counterfactual was correct after all. The bad proof was self-fulfilling. Using quining, one can show such a self-confirming

spurious proof genuinely exists; the situation is once again essentially a soundness sentence, with a specific self-referential witness in place of the existential “there is a proof,” so the spurious conclusion is provable by the same mechanism that made FairBot cooperate. An embedded agent reasoning about a world that contains itself can be led, by valid proofs, into confidently wrong beliefs about what would have happened had it acted otherwise. (Reassuringly, one cannot get a spurious counterfactual about the action one *actually* takes, only about roads not traveled, but that is cold comfort to a planner.) Fixing this requires care about the order in which the agent attends to proofs, and it is one of the threads tying the Löbian material to the decision-theoretic problem of counterfactuals.

9 Takeaways

The durable lesson is that self-reference is not an exotic edge case to be engineered around but a structural feature of any agent that is part of the world it reasons about and that modifies or reproduces itself within that world. The moment an agent’s reasoning has to range over its own future reasoning, or over the reasoning of a successor built in its own image, Löb’s theorem is waiting. It tells us that trust in proofs cannot be bootstrapped for free: a consistent system cannot certify its own soundness, so a proof-using agent cannot straightforwardly trust a successor as strong as itself, and the obvious fixes either bleed the agent’s strength to nothing in finitely many steps or smuggle the successor’s exact actions into the parent’s prediction, violating the very Vingean principle that made a more capable successor worth building.

This is why *reflective stability*, the property of an agent’s commitments surviving its own self-modification, is load-bearing for safety rather than a curiosity. A safety property is only as good as its survival under self-improvement. An agent that would discard its constraints, or could no longer justify them, upon becoming more capable, offers a far weaker guarantee than one whose constraints provably carry across each step of self-improvement. The tiling-agents program is the search for logics in which the good properties are invariant under the agent’s own growth, and the Löbstacle marks exactly where the naive attempts fail. The same self-reference, finally, is double-edged: turned around, it is what lets programs that read each other’s code reach robust cooperation, and it is the hidden source of the spurious counterfactuals that make embedded decision-making hard. Understanding agency in a world where the agent is not a spectator outside the system but a theorem-proving part of it means making peace with Löb.

Sources

- Patrick LaVictoire, *An Introduction to Löb’s Theorem in MIRI Research* (read through Section 3: Introduction; Crash Course in Löb’s Theorem; Direct Uses, including the Löbstacle, Löbian cooperation, and spurious counterfactuals). <https://intelligence.org/files/lob-notes-IAFF.pdf>
- *Vingean reflection* (LessWrong / Arbital tag). <https://www.lesswrong.com/w/vingean-reflection>
- *Walkthrough of the Tiling Agents for Self-Modifying AI paper* (read from the start through “Finite Descent Problem,” plus “What self-modifying agents need”). <https://www.lesswrong.com/posts/QGrX3qK3qxQYK9D4C/walkthrough-of-the-tiling-agents-for-self-modifying-ai-paper>
- The framing of why a logical setting is appropriate (strip empirical uncertainty; idealize agents and environments as programs; proofs are strictly more general than simulation) follows the Iliad Intensive Agency-day lecture slides. Underlying paper: Yudkowsky and Herreshoff, *Tiling Agents for Self-Modifying AI, and the Löbian Obstacle*; a fully formal proof of Löb’s theorem is in Yudkowsky’s *Cartoon Guide to Löb’s Theorem*.